Universidad de San Carlos de Guatemala

Centro Universitario de Occidente

División de Ciencias de la Ingeniería

Sistemas Operativos 2

Ing. Otto Soto

Manual Técnico - Proyecto FInal

Angel Abelino Son Vicente 202030885

Índice

Tecnologías e infraestructura **4**
Estructura del proyecto 5
Arquitectura general **5**
Hilos del sistema **6**
¿Por qué un pool de 10 trabajadores? 6
Terminación 6
Modelo de datos **7**
Pedido 7
Niveles de servicio 7
Tipos de mercancía 7
Recursos 8
Clientes y catálogo 8
Recursos compartidos: semáforos y exclusión mutua **9**
Un semáforo contador por recurso 9
¿Por qué un semáforo y no un candado? 9
¿Por qué un semáforo justo? 9
Tres formas de adquirir 9
Otras primitivas de sincronización 10
Planificación: políticas de atención **11**
Las cinco políticas 11
Cambio de política en caliente 11
La lista de espera 11
El despachador 12
Procesamiento de un pedido **13**
Prevención de deadlock en los pedidos **14**
Comunicación Recepción -> Inventario **14**
El requisito 14
El diseño 15
Garantías 15
Reevaluación automática 16
Exclusión mutua en el inventario 16
Almacenamiento no contiguo **17**
El requisito 17
La analogía con la paginación 17
Cómo funciona 17
Fragmentación 17
Conflicto de recursos: provocar, detectar y resolver deadlock **18**
El escenario 18
Detección con el grafo de espera 18
Resolución manual 19
El mismo escenario con prevención 19

Estrategias frente al deadlock..... 19
Estadísticas..... 20
Tiempo simulado..... 20
API REST..... 21
Interfaz web..... 22
Manejo de excepciones..... 23
Configuración..... 24
Compilación, ejecución y control de versiones..... 25
Resumen de conceptos aplicados..... 26

Tecnologías e infraestructura

El backend está escrito en Java 21, como exige el enunciado. El paquete java.util.concurrent ofrece semáforos, colas bloqueantes, candados y variables de condición reales, que son las herramientas que se usan en todo el sistema.

Spring Boot 3.5 se usa solo con spring-boot-starter-web, que trae un Tomcat embebido. Su papel es exponer la API REST, servir la interfaz e inyectar la configuración, todo desde un único jar. Toda la concurrencia de la simulación está hecha a mano con hilos y primitivas de Java; no se usa ninguna abstracción de Spring para eso.

El proyecto se construye con Maven mediante el wrapper ./mvnw, así que no hace falta instalar Maven. Las pruebas usan JUnit 5.

La interfaz es HTML, CSS y JavaScript sin framework. No necesita compilación: Spring Boot la sirve como recursos estáticos desde src/main/resources/static/.

No hay base de datos. Como es una simulación, todo el estado vive en memoria y se reinicia al reiniciar el servidor.

Para ejecutarlo solo se necesita el JDK 21 o superior, el puerto 8080 libre y un navegador moderno.

Estructura del proyecto

src/main/java/gt/cunoc/redxela/

├── RedXelaApplication.java Punto de entrada (Spring Boot)

├── modelo/ Pedido, Cliente, Producto, Catalogo y enums

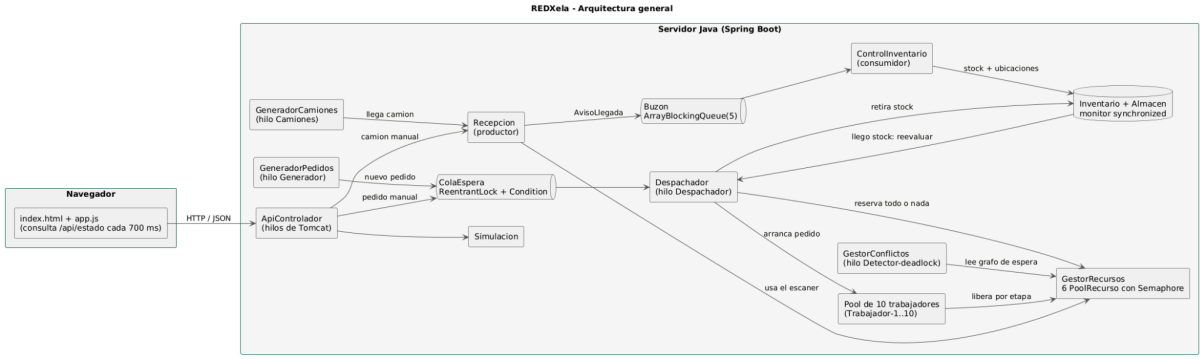
├── recursos/ PoolRecurso (semáforo por recurso), GestorRecursos

├── planificacion/ PoliticaPlanificacion, ColaEspera, Despachador, TrabajadorPedido

Estructura del proyecto

ProyectoFinal/	Contenido
pom.xml	Configuracion de Maven
mvnw	Wrapper de Maven
src/main/java/gt/cunoc/redxela/	Codigo fuente
RedXelaApplication.java	Punto de entrada (Spring Boot)
modelo/	Pedido, Cliente, Producto, Catalogo y enums
recursos/	PoolRecurso (semaforo por recurso), GestorRecursos
planificacion/	PoliticaPlanificacion, ColaEspera, Despachador, TrabajadorPedido
inventario/	Recepcion (productor), ControlInventario (consumidor), Inventario
almacen/	Almacen (mapa de bits), Lote (tabla de ubicaciones), Ubicacion
deadlock/	OperacionConflicto, GrafoEspera, Conflicto, GestorConflictos
estadisticas/	Estadisticas
generador/	GeneradorPedidos, GeneradorCamiones
simulacion/	Simulacion, RelojSimulado, RegistroEventos
api/	ApiControlador, ConstructorEstado, Vistas, ManejadorErrores
excepciones/	RedXelaException y subclases
src/main/resources/	Recursos
application.properties	Parametros de la simulacion
static/	index.html, styles.css, app.js
src/test/java/gt/cunoc/redxela/	NucleoTest, InventarioTest, DeadlockTest
docs/	Manuales, guia de pruebas, capturas y diagramas

Arquitectura general



ApiControlador no contiene lógica de simulación: solo traduce cada petición HTTP a una llamada sobre Simulacion. Los hilos de Tomcat que atienden esas peticiones se comportan como productores adicionales, porque agregan pedidos y camiones o cambian la política mientras la simulación corre. Por eso todo lo que tocan es seguro para hilos.

ConstructorEstado arma una "foto" completa del sistema, que la interfaz pide periódicamente a /api/estado.

Hilos del sistema

Cada hilo tiene un nombre propio, y ese nombre aparece en la bitácora. Así se puede comprobar desde la interfaz qué hilo hizo cada cosa.

- Despachador (uno). Es el planificador: ordena la lista de espera con la política vigente y arranca los pedidos que pueden obtener sus recursos.
- Trabajador-1 a Trabajador-10 (un pool fijo de diez). Cada uno procesa un pedido, etapa por etapa.
- Generador (uno). Crea pedidos aleatorios siguiendo llegadas de Poisson.
- Camiones (uno). Hace llegar camiones con mercadería aleatoria.
- Recepcion (uno). Es el productor: escanea camiones y envía avisos a Inventario.
- Inventario (uno). Es el consumidor: procesa los avisos y actualiza el stock y el almacén.
- Detector-deadlock (uno). Revisa el grafo de espera cada 500 ms.
- OP-n (dos por escenario). Son las operaciones del escenario de deadlock.
- Muestreo (uno, de un ScheduledExecutorService). Mide la ocupación de los recursos cada 250 ms.
- http-nio-8080-exec-* (los de Tomcat). Atienden la API.

¿Por qué un pool de 10 trabajadores?

Cada pedido necesita al menos un encargado de bodega, y solo hay 10. Por lo tanto nunca puede haber más de 10 pedidos en proceso a la vez. Un pool más grande tendría hilos ociosos y uno más pequeño limitaría el paralelismo sin razón. Se usa Executors.newFixedThreadPool para reutilizar los hilos en vez de crear uno nuevo por pedido.

Terminación

Todos los hilos de larga vida revisan una bandera volatile boolean activo y responden a interrupt(). Ninguna espera es infinita sin posibilidad de cancelarse: se usan poll y offer con tiempo límite, o esperas que se pueden interrumpir.

Modelo de datos

Pedido

Un Pedido guarda todo lo que pide el enunciado y algunos datos adicionales:

- un id único, generado con una secuencia atómica;
- el cliente que lo hizo y su nivel de servicio (1 a 5);
- las líneas: producto, cantidad y tipo de mercancía de cada producto;
- el total de artículos, que es la suma de unidades y funciona como estimación del trabajo;
- el estado: EN_COLA, ESPERANDO_STOCK, ESPERANDO_RECURSOS, EN_PROCESO, COMPLETADO o CANCELADO;
- los requerimientos de recursos (sección 4.3);
- los instantes de llegada, inicio y fin, en milisegundos simulados, para calcular espera, proceso y cumplimiento del SLA;
- si fue ingresado manualmente;
- datos de avance: etapa, progreso, tiempo restante, motivo de espera y política con la que fue despachado.

Los datos de identidad (id, cliente, líneas y requerimientos) son final e inmutables, así que cualquier hilo los puede leer sin sincronizar. Los datos de avance los escribe un solo hilo, el trabajador o el despachador, y los lee la API. Por eso basta con declararlos volatile: eso garantiza que los cambios sean visibles entre hilos, y no hace falta exclusión mutua porque nunca hay dos hilos escribiendo el mismo campo.

Niveles de servicio

Los cinco niveles son Exprés (espera máxima de 1 minuto, casi inmediato), Prioritario (10 minutos), Estándar (30), Programado (60) y Económico (120).

El límite de SLA de un pedido es su instante de llegada más la espera máxima de su nivel. Un pedido cumple su SLA si inicia su procesamiento antes de ese límite.

Tipos de mercancía

existen cinco tipos, cada uno con una combinación distinta de recursos. Se definieron seis:

Tipo	Encargado	Montacargas	Escáner	Área carga	Control calidad	Empaque
General	1					1
Frágil	1				1	1
Pesado	1	1		1		

Tipo	Encargado	Montacargas	Escáner	Área carga	Control calidad	Empaque
Electrónico	1		1		1	1
Voluminoso	2	1				1
Perecedero	1			1	1	

Cuando un pedido lleva varios tipos de mercancía, sus requerimientos se combinan tomando el máximo de cada recurso (merge(recurso, cant, Math::max)). El pedido se procesa como una sola orden, así que usa una estación de cada clase y no una por línea.

Existe un escenario realista en que dos pedidos distintos necesiten los mismos dos recursos en momentos cercanos. Con estos tipos ocurre de forma natural: los pedidos Frágiles y Electrónicos compiten por control de calidad y empaque; los Pesados y Perecederos, por encargados y por las únicas dos áreas de carga; y los Electrónicos compiten con Recepción por el único escáner. El sistema resuelve estas contenciones con semáforos y con la reserva todo o nada de la sección 8.

Recursos

TipoRecurso define la capacidad de cada recurso: 10 encargados, 3 montacargas, 1 escáner, 2 áreas de carga, 4 estaciones de control de calidad y 6 de empaque. El orden en que están declarados en el enum es además el orden global de adquisición, que es la base de la prevención de deadlock.

Clientes y catálogo

RegistroClientes guarda los clientes en un ConcurrentHashMap y les asigna ids con un AtomicLong, porque la API puede crear clientes al mismo tiempo que el generador los lee. Viene con cinco clientes precargados, uno por nivel.

Catalogo contiene 15 productos fijos, de P-001 a P-015. Cada uno tiene un tipo de mercancía y un volumen por unidad, que se usa en el almacén. Después de construirse no cambia.

Recursos compartidos: semáforos y exclusión mutua

Un semáforo contador por recurso

Cada recurso se administra con un PoolRecurso:

this.semaforo = new Semaphore(tipo.getCapacidad(), true);   // justo (FIFO)

this.duenios = new String[tipo.getCapacidad()];   // quién tiene cada unidad

Cada permiso del semáforo representa una unidad libre. Tomar n unidades es acquire(n) y devolverlas es release(n): es el semáforo contador de Dijkstra, con sus operaciones P y V. El escáner tiene capacidad 1, así que su semáforo es binario y se comporta como un mutex. Ahí es donde mejor se ve la exclusión mutua: Recepción, los pedidos Electrónicos y las operaciones del escenario de deadlock se turnan para usarlo.

Además del semáforo, el arreglo duenios registra quién tiene cada unidad. La interfaz lo usa para mostrar quién ocupa cada estación, y el detector lo usa para construir el grafo de espera. El arreglo está protegido por el monitor del objeto (synchronized) y se mantiene un invariante: siempre hay al menos tantas casillas vacías como permisos disponibles. Para lograrlo, al adquirir se toma primero el permiso y después la casilla, y al liberar se vacía primero la casilla y después se devuelve el permiso.

¿Por qué un semáforo y no un candado?

Hay dos razones. La primera es que las unidades de un pedido las reserva el hilo Despachador pero las libera el hilo Trabajador, un ReentrantLock o un monitor synchronized solo puede liberarlos el mismo hilo que los tomó, un semáforo no tiene dueño, así que cualquier hilo puede hacer release.

La segunda es que la mayoría de los recursos tienen varias unidades idénticas. Un semáforo contador expresa directamente "hasta 6 estaciones de empaque a la vez", mientras que un candado solo expresa "una a la vez".

¿Por qué un semáforo justo?

Con fair = true, los hilos bloqueados se atienden en orden de llegada. Así ninguno se queda esperando para siempre mientras otros se le adelantan, lo que evita la inanición entre los hilos que esperan un recurso.

Hay un detalle importante: tryAcquire(n) sin tiempo ignora la justicia y se cuela delante de los hilos que ya esperan. Por eso intentarAdquirir usa tryAcquire(n, 0, ms), que sí respeta la fila. Gracias a eso, el despachador nunca le quita unidades a un hilo que ya estaba bloqueado esperando, como Recepción esperando el escáner.

Tres formas de adquirir

PoolRecurso ofrece tres maneras de pedir unidades, cada una pensada para un uso distinto.

intentarAdquirir(duenio, n) no bloquea: toma las n unidades o ninguna. La usa el despachador para los pedidos.

adquirir(duenio, n, timeoutMs) espera como máximo un tiempo límite. Recepción la usa con el escáner y reintenta mientras no lo consigue; así nunca queda bloqueada para siempre y puede revisar entre intentos si debe detenerse.

adquirirBloqueante(duenio, n) espera sin límite, pero se puede interrumpir. La usan las operaciones del escenario de deadlock: esa espera sin límite es la que permite que el deadlock ocurra, y la interrupción es la que permite resolverlo.

Mientras un hilo espera, queda anotado en el mapa esperando (un ConcurrentHashMap). De ese mapa sale la información para construir el grafo de espera.

Otras primitivas de sincronización

Además de los semáforos, el sistema usa varias primitivas de java.util.concurrent, cada una elegida para un problema concreto:

- ReentrantLock con Condition en ColaEspera, para que el despachador duerma hasta que haya algo nuevo, sin espera activa (sección 6.3).
- Monitores synchronized en Inventario, Estadisticas, el arreglo de dueños de PoolRecurso y la lista de conflictos. Son secciones críticas cortas en las que el mismo hilo bloquea y desbloquea, y para eso un monitor es lo más simple y claro.
- ArrayBlockingQueue como buzón acotado entre Recepción e Inventario (sección 9).
- LinkedBlockingQueue como patio de camiones. No tiene límite porque un camión que llega nunca se rechaza.
- CountDownLatch en OperacionConflicto, para que la operación que cede espere a que la ganadora termine.
- AtomicLong para secuencias de ids y contadores. El incremento es atómico mediante una operación CAS, sin necesidad de candado.
- ConcurrentHashMap para clientes, operaciones, pedidos en proceso y registros de espera, porque permite lecturas concurrentes sin bloquear toda la tabla.
- volatile para las banderas de actividad, la política vigente y los datos de avance, donde hay un solo escritor y solo hace falta garantizar visibilidad.

Planificación: políticas de atención

Las cinco políticas

PoliticaPlanificacion es un enum en el que cada valor implementa un método clave(pedido, ahora): es el patrón Strategy. El pedido con la menor clave se atiende primero, y la clave se muestra en la interfaz para que se entienda el criterio de orden.

Prioridad estricta. La clave es el nivel de servicio. Corresponde a la planificación por prioridades sin expropiación. Su ventaja es que los pedidos Exprés siempre salen primero; su desventaja es la inanición: con carga alta, un pedido Económico puede no atenderse nunca.

Prioridad con envejecimiento. La clave es nivel − minutosEspera / 5. Es la técnica de aging: la prioridad de un pedido mejora mientras espera, y cada 5 minutos sube un nivel. Elimina la inanición, a costa de que algún pedido urgente pueda retrasarse un poco.

Menos artículos primero (SJF). La clave es el total de artículos. Es Shortest Job First, usando la cantidad de unidades como duración estimada, tal como indica el enunciado. Minimiza la espera promedio y libera estaciones rápido, pero los pedidos grandes pueden esperar mucho y el nivel de servicio no cuenta.

Plazo más cercano (EDF). La clave son los minutos que le faltan al pedido para vencer su SLA. Es Earliest Deadline First, una política de tiempo real. Maximiza el cumplimiento de plazos mientras la carga es manejable, pero con sobrecarga se degrada: los vencimientos se encadenan en efecto dominó.

Orden de llegada (FIFO). La clave es el minuto de llegada. Es First Come, First Served: simple y justa, aunque no distingue urgencias. Sirve como línea base para comparar las demás.

Cuando dos pedidos tienen la misma clave, se desempata por nivel, luego por orden de llegada y por último por id. Así el orden es total y siempre el mismo.

Cambio de política en caliente

La política vigente es un campo volatile del Despachador. Al cambiarla (PUT /api/politica/{id}) se llama a cola.notificarCambio(), que despierta al despachador, y la siguiente revisión ya usa el nuevo orden. No se detiene ni se reinicia nada.

Cada pedido guarda con qué política fue despachado. Gracias a eso, las estadísticas se pueden separar por política y comparar entre sí.

La lista de espera

La forma obvia de implementar una lista ordenada por prioridad sería una PriorityBlockingQueue, pero no sirve aquí. Una cola de prioridad es un montículo que ordena al insertar, y con la política de envejecimiento la clave de cada pedido cambia con el

tiempo; el montículo no se reordena solo. Además, cada cambio de política obligaría a reconstruir la cola entera.

Por eso ColaEspera guarda una lista simple protegida por un ReentrantLock. En cada revisión, el despachador obtiene una copia y la ordena con la política vigente. El ordenamiento se hace fuera del candado, para no bloquear a los productores (el generador y la API) mientras tanto.

Para no consumir CPU revisando la lista sin parar, el despachador espera en una variable de condición: hayCambio.await(500 ms). Lo despiertan con signalAll() cuando llega un pedido, se libera un recurso, llega stock o cambia la política. El límite de 500 ms asegura que el envejecimiento avance aunque no ocurra ningún evento. Una bandera cambio evita perder un aviso que llegue justo antes del await; es el patrón clásico para evitar el problema del lost wake-up.

El despachador

En cada revisión, el despachador recorre la lista ya ordenada y, para cada pedido:

1. Si falta stock de algún producto, lo marca como ESPERANDO_STOCK, anota qué falta y pasa al siguiente.
2. Si hay stock, intenta reservar todos sus recursos. Si lo logra, retira el stock, lo quita de la lista y lo envía al pool de trabajadores.
3. Si no consigue los recursos, lo marca como ESPERANDO_RECURSOS, anota cuáles faltan y pasa al siguiente.

El despachador no se detiene en el primer pedido que no puede iniciar. Si el primero de la fila necesita un montacargas y no hay, el siguiente, que quizá solo necesita una estación de empaque, sí puede arrancar. Así no se desperdician recursos libres. Es el mismo principio por el que un planificador no deja la CPU ociosa cuando el proceso de mayor prioridad está bloqueado.

Solo el despachador retira existencias. Por eso, entre verificar el stock y retirarlo nadie más puede consumirlo, y no existe la condición de carrera de "verificar y luego actuar". Si aun así algo fallara después de reservar los recursos, se liberan.

El motivo de espera queda guardado en el pedido y se muestra en la interfaz, para que se entienda por qué cada pedido sigue esperando.

Procesamiento de un pedido

TrabajadorPedido corre en un hilo del pool y recibe el pedido con todos sus recursos ya reservados.

Como pide el enunciado, la cantidad de trabajo es proporcional a las unidades:

tiempo total = totalArticulos × msPorUnidad (250 ms por unidad por defecto)

Ese tiempo se reparte entre las etapas que aplican al pedido, según su peso. La recolección en bodega (35 % del tiempo) usa montacargas y escáner. El empaque (25 %) usa las estaciones de empaque. El control de calidad (20 %) usa sus estaciones. La carga y despacho (20 %) usa las áreas de carga. La recolección siempre aplica; las demás etapas solo si el pedido necesita alguno de sus recursos.

Al terminar cada etapa se liberan de inmediato sus recursos, y otros pedidos pueden usarlos mientras este sigue con las etapas siguientes. Al final se libera lo que quede, como los encargados. Esta liberación temprana no puede causar deadlock, porque el trabajador solo suelta recursos: nunca pide nada nuevo a mitad del procesamiento.

Para mostrar el avance, el trabajador duerme en pasos de 100 ms y actualiza el progreso y el tiempo restante del pedido. Esos campos son volatile, y la interfaz los lee para dibujar la barra de progreso.

Todo el trabajo está dentro de un try … finally { gestor.liberarTodo(duenio) }. Aunque ocurra una excepción o una interrupción, ningún recurso queda tomado. Sin esa garantía, una fuga de permisos iría dejando al sistema sin recursos con el tiempo.

Prevención de deadlock en los pedidos

Según Coffman, un deadlock solo puede ocurrir si se cumplen las cuatro condiciones a la vez: exclusión mutua, retener y esperar, no expropiación y espera circular. Basta con romper una para que el deadlock sea imposible.

Los pedidos normales rompen dos de ellas, así que por diseño nunca pueden caer en deadlock.

Rompen "retener y esperar" con una reserva todo o nada. GestorRecursos.intentarReservar intenta tomar todos los recursos del pedido sin bloquearse. Si falta alguno, devuelve los que ya había tomado y el pedido vuelve a la lista de espera sin retener nada.

Rompen "espera circular" con un orden global de adquisición. Los recursos se piden siempre en el orden del enum TipoRecurso: encargado, montacargas, escáner, área de carga, control de calidad y empaque. Con un orden total entre los recursos no se puede formar un ciclo.

La reserva todo o nada bastaría por sí sola. El orden global se mantiene como defensa adicional y porque es el mismo mecanismo de prevención que se muestra en el escenario de conflicto (sección 11.4), lo que permite comparar ambos casos.

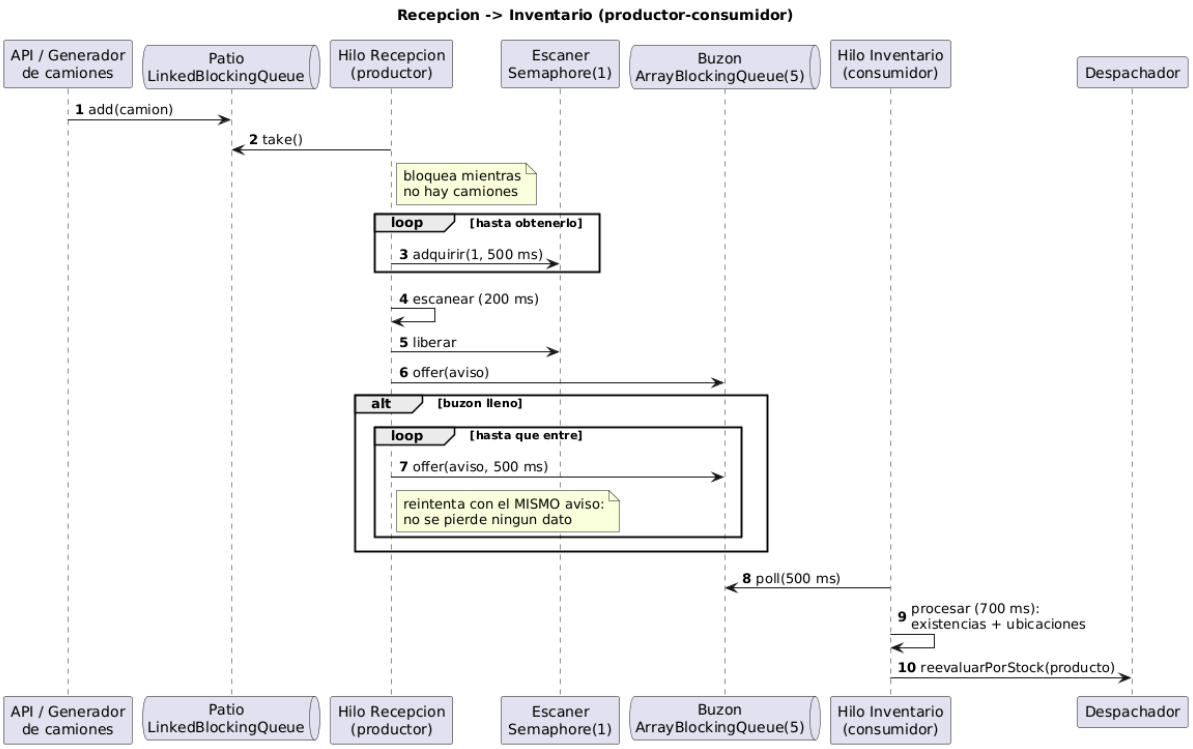
El costo de esta prevención es que un pedido que necesita muchos recursos distintos puede esperar más, porque no los va acumulando de a poco. Se compensa revisando a todos los pedidos de la fila (sección 6.4) y liberando los recursos por etapa (sección 7).

Comunicación Recepción -> Inventario

El requisito

Recepción e Inventario son áreas independientes que no deben modificar las estructuras internas de la otra. Deben comunicarse enviándose mensajes, y la comunicación tiene que funcionar aunque Recepción produzca más rápido de lo que Inventario consume, sin perder datos y sin quedar bloqueada indefinidamente. Es el problema clásico del productor-consumidor con búfer acotado.

El diseño



El mensaje es un AvisoLlegada: un record inmutable con id, producto, cantidad e instante. Al ser inmutable, puede pasar de un hilo a otro sin sincronización adicional.

El búfer es un ArrayBlockingQueue de capacidad 5 (configurable). Internamente usa un ReentrantLock con dos condiciones, notFull y notEmpty, que es exactamente la solución clásica del problema con monitores.

El productor, Recepción, toma el escáner compartido, escanea la mercadería y deja el aviso en el buzón. Nunca toca las existencias ni el almacén.

El consumidor, Inventario, procesa los avisos más lento que Recepción a propósito (700 ms contra 200 ms), para que el buzón se llene y el efecto se pueda observar en la interfaz.

Garantías

No se pierden datos. Si el buzón está lleno, Recepción no descarta el aviso: reintenta offer(aviso, 500 ms) con el mismo aviso hasta que entra. Del lado del almacén, lo que no cabe queda como "pendiente de ubicar" y se ubica cuando se libera espacio.

No hay bloqueo indefinido. En lugar de put y take, que esperan para siempre, se usan offer y poll con tiempo límite. Entre un intento y otro, cada hilo revisa si debe detenerse, así que siempre puede terminar. Aunque el consumidor dejara de funcionar, el productor no quedaría colgado.

La interfaz muestra cuántas veces se llenó el buzón. El contador aumenta cada vez que el primer offer, sin espera, falla.

Reevaluación automática

Cuando Inventario.registrarEntrada aumenta el stock de un producto, ejecuta un callback que llama a Despachador.reevaluarPorStock(producto). Este método devuelve a EN_COLA a los pedidos que esperaban ese producto y despierta al despachador.

El callback se ejecuta fuera del candado de Inventario. Retener un candado mientras se llama a código de otro componente es una fuente clásica de deadlocks, y así se evita.

Exclusión mutua en el inventario

Inventario es un monitor: todos sus métodos públicos son synchronized, de modo que las existencias, el almacén y los pendientes se modifican juntos y de forma atómica. Almacen no es seguro para hilos por sí mismo, pero su único dueño es Inventario, que lo protege con su propio candado.

Retirar el stock de un pedido también es todo o nada: primero se verifica que alcancen todos los productos y solo entonces se retira. Si algo no alcanza, se lanza StockInsuficienteException y no se retira nada.

Almacenamiento no contiguo

El requisito

Un lote puede necesitar varias ubicaciones del almacén, y esas ubicaciones no tienen por qué ser contiguas: puede quedar repartido en distintos pasillos y niveles, según el espacio libre al momento de recibirlo.

La analogía con la paginación

El almacén se diseñó como un sistema de paginación de memoria.

El almacén físico equivale a la memoria física. Está dividido en 240 ubicaciones de tamaño fijo (6 pasillos × 4 niveles × 10 posiciones, de 10 unidades de volumen cada una), que hacen el papel de los marcos. Cada lote es como un proceso, y cada ubicación que ocupa es como una de sus páginas.

Cada Lote tiene su propia tabla de ubicaciones, una lista de índices que cumple la función de la tabla de páginas. Las ubicaciones libres se llevan en un mapa de bits (BitSet), igual que la tabla de marcos libres.

Cómo funciona

Al guardar un lote, primero se calcula cuántas unidades caben en una ubicación según el volumen del producto (10 / volumenUnidad, con un mínimo de 1). Luego se recorren las ubicaciones libres con nextClearBit, siguiendo la estrategia de primer ajuste, y se van llenando sin importar si están juntas. Cada índice usado se anota en la tabla del lote.

Al retirar mercadería para un pedido, se toma primero del lote más antiguo, como en una rotación de inventario. Las ubicaciones que quedan vacías se marcan como libres en el mapa de bits y se quitan de la tabla del lote, y un lote que se queda sin ubicaciones desaparece.

Fragmentación

No hay fragmentación externa, porque cualquier ubicación libre sirve, igual que cualquier marco libre sirve para una página. Sí hay fragmentación interna: la última ubicación de un lote puede quedar a medio llenar, como la última página de un proceso.

Con el uso, los espacios vacios que dejan los retiros se reutilizan, y los lotes nuevos quedan dispersos por el almacén. Eso se ve en la interfaz al hacer clic en un lote.

Se descartó la asignación contigua porque exigiría encontrar un bloque continuo del tamaño del lote. Con el tiempo aparecería fragmentación externa: habría espacio libre suficiente en total, pero no junto, y habría que rechazar mercadería o mover lotes para compactar. La asignación no contigua evita ambos problemas.

Conflicto de recursos: provocar, detectar y resolver deadlock

El escenario

El enunciado pide un caso en que dos operaciones necesiten los mismos dos recursos, pero en orden inverso. Se eligió este:

- Recepción de camión pesado: primero toma el escáner para leer la guía del camión, y después pide un montacargas para bajar la carga.
- Despacho extra pesado: primero toma los 3 montacargas para levantar la carga, y después pide el escáner para verificar la guía de salida.

A diferencia de los pedidos, OperacionConflicto toma los recursos uno por uno con espera bloqueante (adquirirBloqueante) y retiene cada uno mientras trabaja en ese paso, durante 1,5 segundos.

Si ambas arrancan al mismo tiempo, la Recepción se queda con el único escáner esperando un montacargas, y el Despacho se queda con los tres montacargas esperando el escáner. Se cumplen las cuatro condiciones de Coffman:

- hay exclusión mutua, porque los recursos se controlan con semáforos;
- cada una retiene y espera, porque conserva lo que tomó en su primer paso;
- no hay expropiación, porque nadie le quita recursos a otro;
- hay espera circular, porque cada una espera a la otra.

No es una animación: las dos operaciones son hilos reales, bloqueados en Semaphore.acquire.

Detección con el grafo de espera

El hilo Detector-deadlock revisa el sistema cada 500 ms.

Primero construye el grafo de espera (wait-for graph). Para cada recurso, traza una arista A -> B por cada hilo A que lo está esperando y cada dueño B que lo tiene. La información sale de los registros esperando y duenios de cada PoolRecurso.

Después busca un ciclo con una búsqueda en profundidad (DFS), recordando los nodos del camino actual. Si una arista llega a un nodo que ya está en el camino, las aristas desde ese nodo hasta el final del camino forman el ciclo. La búsqueda es O(V + E).

La foto del grafo no es atómica, porque se lee recurso por recurso, y en un instante de transición podría aparecer un ciclo que en realidad no existe. Por eso el conflicto solo se declara si el mismo ciclo aparece en dos revisiones seguidas. Un deadlock real no cambia con el tiempo; una transición sí.

Al confirmarlo, se registra un Conflicto con las operaciones que participan, qué retiene cada una, qué espera y quién lo tiene, los recursos involucrados y un texto que explica las condiciones de Coffman. Primero se marcan las operaciones como BLOQUEADA y

después se publica el conflicto, para que la interfaz nunca muestre un conflicto cuyas operaciones todavía no aparecen bloqueadas. Ese orden corrigió una condición de carrera que se detectó en las pruebas.

Se eligió detectar el deadlock, y no solo prevenirlo, para poder mostrar un deadlock real con su grafo y sus condiciones, en vez de solo afirmar que no ocurre.

Resolución manual

El usuario elige qué operación continúa (POST /api/conflicto/{id}/resolver). La resolución es una expropiación controlada y sigue estos pasos:

1. Se valida que el conflicto exista, que no esté resuelto y que la operación elegida participe en él, y se marca como resuelto.
2. A la operación que debe ceder se le indica a quién cede y se interrumpe su hilo con interrupt(). Como estaba bloqueada en acquire, que es interrumpible, sale con InterruptedException.
3. La operación que cede libera todo lo que tenía. La ganadora obtiene el recurso que esperaba, porque el semáforo la despierta.
4. La operación que cede pasa a EN_LISTA_ESPERA y espera a que la ganadora termine, con CountDownLatch.await().
5. Cuando la ganadora termina, la otra reintenta, esta vez pidiendo los recursos en el orden global. Su tarjeta muestra "intento 2" y ya no puede volver a caer en el mismo ciclo.

Así se cumple el requisito de que la operación que no continúa regrese a la lista de espera para atenderse más adelante.

Se eligió interrumpir el hilo en vez de liberar sus permisos desde fuera, porque liberar los recursos de un hilo que sigue creyendo que los tiene rompería la consistencia. Con la interrupción, es el propio hilo el que suelta lo suyo de forma ordenada en su catch y su finally. Es la forma cooperativa y segura de cancelar un hilo en Java.

El mismo escenario con prevención

Con la opción "Ejecutar con prevención", ambas operaciones ordenan sus pasos según el orden global de TipoRecurso, en el que el montacargas va antes que el escáner. Las dos piden primero montacargas y luego escáner: una espera a la otra, pero el ciclo no puede formarse, y ambas terminan en el primer intento. Es la misma técnica que usan los pedidos normales (sección 8).

Estrategias frente al deadlock

En el sistema conviven dos de las estrategias clásicas. La prevención, que rompe las condiciones de retener y esperar y de espera circular, se usa en los pedidos normales y en el escenario con prevención. La detección con recuperación, mediante el grafo de espera y la expropiación, se usa en el escenario sin prevención.

La tercera estrategia, la evasión con el algoritmo del banquero, no se usó. Requiere conocer de antemano cuánto va a pedir como máximo cada proceso y es costosa de evaluar en cada asignación. La prevención por orden global es más simple y suficiente para este sistema.

Estadísticas

Estadisticas es un monitor (synchronized) que acumula, por política y por nivel de servicio, los datos de cada pedido completado: cuánto esperó (de la llegada al inicio), cuánto tardó en procesarse (del inicio al fin) y si cumplió su SLA.

La ocupación de los recursos se mide aparte. El hilo Muestreo toma cada 250 ms una muestra de cuántas unidades de cada recurso están en uso y la suma al promedio de la política vigente. Así se obtiene la ocupación promedio de cada recurso con cada política.

Como cada pedido se atribuye a la política con la que fue despachado, se puede dejar correr cada política un rato y compararlas en la misma tabla: completados, espera promedio, procesamiento promedio, porcentaje de SLA cumplido y, por cada nivel, su espera y su cumplimiento.

Tiempo simulado

Los tiempos objetivo del enunciado van de 10 a 120 minutos, demasiado para observarlos en una demostración. Por eso RelojSimulado acelera el tiempo: ahora = msReales × factor.

El factor por defecto es 60, así que 1 segundo real equivale a 1 minuto simulado y un pedido Económico vence a los 2 minutos reales.

La duración del trabajo (msPorUnidad) se mide en milisegundos reales. El reloj convierte los instantes a tiempo simulado para calcular esperas y SLA y para la bitácora. Las políticas también usan el reloj simulado: el envejecimiento sube un nivel cada 5 minutos simulados, es decir, cada 5 segundos reales.

API REST

La API vive en http://localhost:8080/api y todas las respuestas son JSON.

Método	**Ruta**	**Cuerpo o parámetros**	**Descripción**
GET	/estado	—	Foto completa del sistema.
GET	/catalogo	—	Productos, tipos de mercancía, niveles y políticas.
GET	/clientes	—	Lista de clientes.
POST	/clientes	{nombre, correo?, nivel}	Crea una cuenta (201).
POST	/pedidos	{clienteId, nivel?, productos: {"P-001": 3}}	Pedido manual (201).
POST	/recepcion	{codigo, cantidad}	Llega un camión.
POST	/recepcion/rafaga	?camiones=12 (1 a 100)	Llegan N camiones aleatorios.
PUT	/politica/{id}	id de la política	Cambia la política en caliente.
PUT	/generador/pedidos	{activo?, intervaloMs?}	Controla el generador de pedidos.
PUT	/generador/camiones	{activo?, intervaloMs?}	Controla el generador de camiones.
POST	/conflicto/provocar	?prevencion=false|true	Lanza el escenario de conflicto.
POST	/conflicto/{id}/resolver	{ganadora}	Resuelve un conflicto.
POST	/conflicto/limpiar	—	Quita las operaciones terminadas.

Los ids de política son PRIORIDAD_ESTRICTA, ENVEJECIMIENTO, MENOS_ARTICULOS, PLAZO_MAS_CERCANO y FIFO. Si un pedido no indica nivel, se usa el que tiene contratado el cliente.

La interfaz consulta /api/estado cada 700 ms en lugar de usar WebSocket. Es más simple, no requiere dependencias adicionales y alcanza para un "tiempo real" a escala humana. Para armar la foto, cada componente entrega copias de sus datos, así que la consulta no bloquea la simulación más que un instante.

Interfaz web

La interfaz son tres archivos (index.html, styles.css y app.js) que Spring Boot sirve como recursos estáticos, sin ningún framework.

Un setInterval llama a /api/estado cada 700 ms y vuelve a dibujar la pestaña activa. Una bandera evita que se acumulen peticiones si alguna tarda en responder.

La interfaz está dividida en cinco pestañas:

- Operación: recursos, pedidos en proceso, lista de espera, pedido manual, alta de clientes y últimos terminados.
- Inventario y almacén: flujo productor-consumidor, mapa del almacén y lotes.
- Conflicto de recursos: escenario, conflicto activo, operaciones, grafo de espera e historial.
- Estadísticas: indicadores, comparación de políticas y ocupación de recursos.
- Bitácora: últimos eventos con el hilo que los generó.

Todo texto que viene del servidor pasa por una función de escape de HTML, para que, por ejemplo, el nombre de un cliente no pueda inyectar código en la página. Los errores que devuelve la API se muestran como notificaciones.

Manejo de excepciones

El sistema define su propia jerarquía de excepciones, todas derivadas de RedXelaException, que a su vez es una RuntimeException:

- DatosInvalidosException, para entradas inválidas: cantidad no positiva, nombre vacío, política desconocida, conflicto ya resuelto, etc.
- ClienteNoEncontradoException, cuando no existe el id de cliente.
- ProductoNoEncontradoException, cuando no existe el código de producto.
- StockInsuficienteException, cuando se intenta retirar más de lo que hay; en ese caso no se retira nada.

En la API, ManejadorErrores (@RestControllerAdvice) convierte cada excepción en un JSON {"error": "..."} con el código HTTP adecuado: 404 si algo no existe, 400 si los datos son inválidos o el JSON está mal formado, y 500 para cualquier error inesperado, que además se registra en el log.

En los hilos se siguen cuatro reglas:

- Un error no detiene a un hilo de larga vida. El ciclo principal del despachador, Recepción, Inventario y el detector captura cualquier RuntimeException, la registra en la bitácora y sigue trabajando.
- InterruptedException siempre se respeta. Se restaura la marca de interrupción con Thread.currentThread().interrupt() y el hilo termina, o cede, en el caso del deadlock. Nunca se ignora en silencio.
- Los recursos se liberan siempre en un finally, tanto en TrabajadorPedido como en OperacionConflicto.
- Si falla el retiro de stock después de reservar los recursos, los recursos se liberan.

Configuración

Los parámetros están en src/main/resources/application.properties. Cualquiera se puede cambiar al ejecutar, con --clave=valor.

- server.port (8080): puerto HTTP.
- redxela.reloj.factor (60): minutos simulados por minuto real.
- redxela.trabajo.ms-por-unidad (250): milisegundos reales de trabajo por unidad del pedido.
- redxela.generador.intervalo-ms (1500): tiempo medio entre pedidos automáticos.
- redxela.generador.auto-iniciar (false): si el generador arranca encendido.
- redxela.recepcion.ms-escaneo (200): duración del escaneo de un camión.
- redxela.inventario.ms-procesamiento (700): lo que tarda Inventario en procesar un aviso.
- redxela.inventario.capacidad-buzon (5): capacidad del buzón.
- redxela.camiones.intervalo-ms (4000): tiempo medio entre camiones automáticos.
- redxela.conflicto.ms-por-paso (1500): duración de cada paso de las operaciones de conflicto.

El generador de pedidos calcula el tiempo entre llegadas con una distribución exponencial, −media · ln(U). Es el modelo de llegadas de Poisson, el estándar para clientes que llegan de forma independiente. El nivel de cada pedido se sortea con estas probabilidades: Exprés 10 %, Prioritario 20 %, Estándar 35 %, Programado 20 % y Económico 15 %. Cada pedido lleva de 1 a 3 productos, con 1 a 8 unidades cada uno.

Compilación, ejecución y control de versiones

```
./mvnw spring-boot:run              # ejecutar en desarrollo

./mvnw package                      # compilar y probar → target/redxela-1.0.0.jar

java -jar target/redxela-1.0.0.jar  # ejecutar el jar
```

El repositorio sigue un Git Flow simplificado para un solo desarrollador. La rama main tiene la versión entregable y develop es la rama de integración. Cada parte se desarrolló en su propia rama de funcionalidad: feature/nucleo (modelo, recursos, planificación, despachador y trabajadores), feature/inventario (recepción, inventario y almacén), feature/deadlock (escenario de conflicto, detección y resolución) y feature/interfaz (API REST, interfaz web y documentación). Cada una se integró a develop mediante un pull request.

Resumen de conceptos aplicados

Hilos y sincronización real. El sistema tiene nueve clases de hilos con nombre propio, coordinados con Semaphore, ReentrantLock y Condition, monitores synchronized, colas bloqueantes, CountDownLatch y variables atómicas.

Semáforos y exclusión mutua. Cada recurso es un semáforo contador justo; el escáner, con capacidad 1, es un semáforo binario que actúa como mutex.

Planificación. Cinco políticas intercambiables en caliente (prioridad, envejecimiento, SJF, EDF y FIFO), con la cantidad de unidades como estimación del trabajo y estadísticas separadas por política para compararlas.

Prevención de deadlock. Los pedidos reservan todo o nada y piden los recursos en un orden global, lo que rompe dos condiciones de Coffman.

Detección y recuperación de deadlock. Un escenario provoca un deadlock real, que se detecta con un grafo de espera y una búsqueda en profundidad, se explica en la interfaz y se resuelve por expropiación: la operación que cede es interrumpida, libera todo, vuelve a la lista de espera y reintenta en orden seguro.

Comunicación entre procesos. Recepción e Inventario se comunican por paso de mensajes a través de un búfer acotado, sin perder datos ni bloquearse indefinidamente.

Administración de almacenamiento. El almacén usa asignación no contigua, análoga a la paginación: un mapa de bits de ubicaciones libres, una tabla de ubicaciones por lote y asignación por primer ajuste.

Calidad del código. Cada clase está documentada con Javadoc, hay una jerarquía propia de excepciones, los errores de la API se traducen a respuestas claras y los recursos siempre se liberan en bloques finally.